

Password Manager

a. Oggetto

Il progetto consiste nello sviluppo di un'applicazione Python per la gestione sicura di credenziali personali.

L'applicazione permette di salvare login composti da titolo, username, password, sito web e categoria.

Il programma usa una master password per proteggere la cassaforte e cifra username e password prima del salvataggio nel database.

L'interfaccia grafica e realizzata con Tkinter e i dati vengono conservati in un database SQLite locale.

b. Scopo

Lo scopo del progetto e realizzare un password manager semplice ma completo, utile per applicare diversi argomenti studiati durante il corso:

- programmazione in Python
- funzioni e moduli
- paradigma OOP
- gestione degli errori
- interfacce grafiche con Tkinter
- database SQLite
- uso di librerie esterne
- crittografia dei dati
- organizzazione del codice in piu file

L'utente puo creare una cassaforte, accedere con master password, aggiungere credenziali, modificarle, eliminarle, organizzarle per categoria e generare password sicure.

c. Analisi tecnica

Struttura dei file

```
password-manager-python/  
■■■ main.py  
■■■ ui.py  
■■■ database.py  
■■■ crypto_utils.py  
■■■ requirements.txt  
■■■ README.md
```

Ruolo dei file

- ``main.py``: punto di ingresso del programma. Avvia l'interfaccia grafica.
- ``ui.py``: contiene la classe ``PasswordManagerApp``, cioè tutta la grafica Tkinter e il collegamento tra pulsanti, database e crittografia.
- ``database.py``: contiene le funzioni per creare e interrogare il database SQLite.
- ``crypto_utils.py``: contiene le funzioni di crittografia, decrittografia e generazione password.
- ``requirements.txt``: contiene la libreria esterna necessaria, cioè ``cryptography``.

Librerie usate

- ``tkinter``: libreria standard di Python per creare l'interfaccia grafica.
- ``sqlite3``: libreria standard di Python per usare database SQLite.
- ``pathlib``: usata per costruire il percorso del database in modo portatile.
- ``os``: usata per generare il salt casuale e gestire alcuni percorsi.
- ``base64``: usata per trasformare la chiave in un formato compatibile con Fernet.
- ``secrets``: usata per generare password casuali in modo più adatto alla sicurezza.
- ``string``: usata per ottenere lettere, numeri e simboli.
- ``cryptography``: libreria esterna usata per derivare chiavi e cifrare i dati.

Database SQLite

Il database si chiama ``password_manager.db`` e viene creato automaticamente al primo avvio.

Sono presenti tre tabelle:

```

impostazioni
- chiave
- valore

categorie
- id
- nome

password
- id
- titolo
- username_criptato
- password_criptata
- sito
- categoria_id

```

La tabella ``impostazioni`` conserva il salt e il token di verifica della master password.

La master password non viene salvata in chiaro nel database.

La tabella ``categorie`` contiene le categorie create dall'utente.

La tabella ``password`` contiene i login salvati. Username e password vengono inseriti già criptati.

Crittografia

La crittografia è gestita nel file ``crypto_utils.py``.

Il procedimento è questo:

```
master password + salt
    |
    v
PBKDF2HMAC con SHA256
    |
    v
chiave Fernet
    |
    v
cifratura e decifratura dei dati
```

La master password serve per generare una chiave tramite ``PBKDF2HMAC``.

Il salt viene generato casualmente con ``os.urandom(16)`` e salvato nel database.

Per verificare la master password non viene salvata la password, ma un token criptato:

```
TOKEN_VERIFICA = "main_pass_ok"
```

Quando l'utente accede, il programma prova a decriptare il token.

Se la decrittazione riesce e il token corrisponde, la password è corretta.

Generatore di password

Il generatore usa la libreria ``secrets``, più adatta di ``random`` per valori legati alla sicurezza.

L'utente può scegliere:

- lunghezza
- uso di maiuscole
- uso di minuscole
- uso di numeri
- uso di simboli
- numero minimo di numeri
- numero minimo di simboli

Il programma controlla che i criteri scelti siano validi.

Per esempio, non permette di richiedere numeri minimi se i numeri sono disattivati.

Interfaccia grafica

La grafica è realizzata con una classe:

```
class PasswordManagerApp:
```

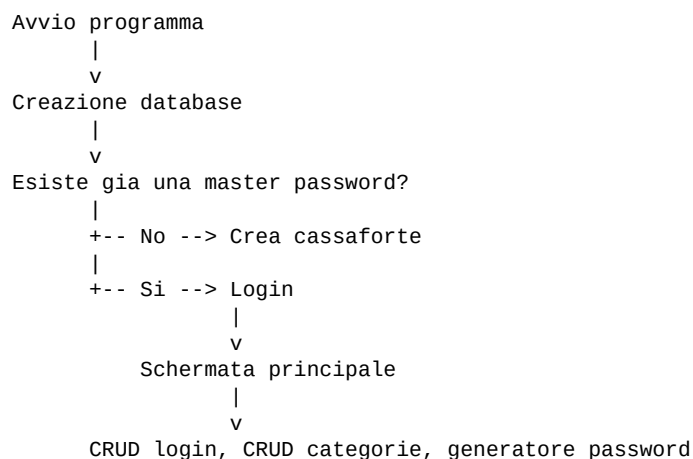
Questa classe gestisce:

- schermata di creazione cassaforte
- schermata di accesso
- schermata principale
- finestra aggiunta/modifica login
- finestra categorie
- finestra generatore password

La schermata principale mostra una tabella con i login salvati.

La password non viene mostrata direttamente nella tabella: viene visualizzata solo quando l'utente seleziona un login e preme `Visualizza` oppure `Modifica`.

Flusso principale



d. Commenti su procedure implementate

Master password

La master password e la parte piu importante del progetto.

Non viene mai salvata direttamente nel database.

Viene usata solo per generare la chiave di cifratura.

Il programma salva:

- salt
- token di verifica criptato

Questo permette di controllare se la master password inserita e corretta senza conoscere o salvare la password originale.

Salvataggio credenziali

Quando l'utente salva un login:

1. inserisce titolo, username, password, sito e categoria
2. username e password vengono criptati
3. il database riceve solo i valori criptati

Quindi, aprendo direttamente il file SQLite, non si leggono le password in chiaro.

Categorie

Le categorie permettono di organizzare i login.

Se una categoria viene eliminata, i login collegati non vengono cancellati.

Il programma imposta semplicemente `categoria\_id` a `NULL`, lasciando quei login senza categoria.

Compatibilit  Linux e Windows

Il programma usa librerie Python portabili.

Su Linux   necessario avere Tkinter installato tramite pacchetto di sistema, per esempio:

```
sudo apt install python3-tk
```

Su Windows il progetto puo essere avviato usando direttamente Python o una virtual environment.

e. Guida utente: input e output

Avvio su Linux

Installa i pacchetti necessari:

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv python3-tk git
```

Scarica il progetto:

```
git clone https://github.com/snuke704/password-manager-python.git  
cd password-manager-python
```

Crea e attiva l'ambiente virtuale:

```
python3 -m venv .venv  
source .venv/bin/activate
```

Installa le dipendenze:

```
python -m pip install -r requirements.txt
```

Avvia il programma:

```
python main.py
```

Avvio su Windows

Da PowerShell:

```
python -m venv .venv  
.\.venv\Scripts\python.exe -m pip install -r requirements.txt  
.\.venv\Scripts\python.exe main.py
```

Primo avvio

Input richiesto:

- master password
- conferma master password

Output atteso:

- messaggio di conferma
- apertura della schermata principale

La master password deve avere:

- almeno 10 caratteri
- almeno una maiuscola
- almeno una minuscola
- almeno un numero
- almeno un simbolo

Accesso successivo

Input richiesto:

- master password

Output atteso:

- se corretta, apertura della cassaforte
- se errata, messaggio di errore

Aggiunta login

Input richiesto:

- titolo
- username

- password
- sito
- categoria

Output atteso:

- il login appare nella tabella principale
- username e password vengono salvati criptati

Modifica login

Input richiesto:

- selezione di un login dalla tabella
- nuovi dati da salvare

Output atteso:

- aggiornamento del login nel database
- aggiornamento della tabella

Eliminazione login

Input richiesto:

- selezione di un login
- conferma eliminazione

Output atteso:

- rimozione del login dal database

Gestione categorie

Input richiesto:

- nome categoria da aggiungere
- nuovo nome categoria
- conferma eliminazione

Output atteso:

- aggiornamento delle categorie disponibili nei filtri e nei form

Generatore password

Input richiesto:

- lunghezza
- criteri sui caratteri
- minimi richiesti

Output atteso:

- password casuale generata
- possibilita di usarla direttamente nel form di inserimento login

f. Conclusioni

Il progetto dimostra l'uso pratico di Python per realizzare un'applicazione completa con interfaccia grafica, database e sicurezza dei dati.

Le funzionalita richieste sono state implementate:

- master password
- crittografia delle credenziali
- database SQLite
- gestione login
- gestione categorie
- generatore password
- interfaccia grafica Tkinter
- compatibilita con GNU/Linux

Il progetto puo essere migliorato in futuro aggiungendo esportazione dei dati, backup cifrato, ricerca avanzata e blocco automatico della cassaforte dopo inattivita.

Note finali

Il file `password\_manager.db` non viene caricato su GitHub perche contiene dati locali.

Il database viene creato automaticamente all'avvio.